REPORT: Assignment 3 Programming Paradigms (Chapters 3–5)

Name: Nguyen Tai Dung

TROY ID: 1695191

---

1. Introduction

This report presents an exploration of three major programming paradigms:

- Functional Programming (Chapter 3)
- Logic Programming (Chapter 4)
- Object-Oriented Programming (Chapter 5)

For each paradigm, representative code examples are selected from the textbook, slightly adapted if necessary, and analyzed in terms of purpose, behavior, and execution.

---

2. Functional Programming (Chapter 3)

2.1 Overview

Functional programming emphasizes:

- Use of functions as primary building blocks
- Recursion instead of loops
- Immutable data

2.2 Example 1: Tail vs Non-Tail Recursion (Scheme)

Code

```
; Non-tail recursion
(define factorial
  (lambda (n)
    (if (= n 1)
        1
        (* n (factorial (- n 1))))))

; Tail recursion
(define factorial
 (lambda (n result)
  (if (= n 1)
      result
      (factorial (- n 1) (* n result)))))
```

Purpose

Compute the factorial of a number and compare two recursion styles.

Description

This program defines two implementations of the factorial function using recursion.

In the **non-tail recursive version**, the function calls itself and then multiplies the result afterward. This means each recursive call must wait for the result of the next call before completing. As a result, multiple stack frames are kept in memory simultaneously, leading to **linear space complexity O(n)**.

In contrast, the **tail-recursive version** carries an accumulator (`result`) that stores intermediate results. The recursive call is the final operation, allowing the compiler or interpreter to optimize the recursion into an iterative process (called **tail call optimization**). This significantly reduces memory usage to **constant space O(1)** in optimized environments.

How to run

Environment: Install a Scheme interpreter such as **Racket, MIT Scheme or an online compiler**

Steps:

1. Open the interpreter or online compiler
2. Copy the code into a file (e.g., `factorial.scm`)
3. Run the program

The code provided in the textbook only describes the function definitions and illustrates the programming concept. To obtain actual results, an additional main function is required to execute the program and display output.

```
(display (factorial 6))
(newline)
(display (factorial 6 1))
```

Expected Output:

```
720
720
```

2.3 Example 3: Haskell Functions and Currying

Code

```
fact 0 = 1
fact n = n*fact (n-1)

plus2 = (2+)
times3 = (*3)
```

Purpose

Demonstrate function definition and currying.

Description

The function `fact` is defined using pattern matching, where the base case (`fact 0 = 1`) terminates recursion and the recursive case reduces the problem size. This reflects the declarative nature of Haskell, where functions are defined by rules rather than step-by-step instructions.

The expressions `plus2 = (2+)` and `times3 = (*3)` illustrate **currying**, a technique in which a function with multiple arguments is transformed into a sequence of single-argument functions. In this case, `(2+)` creates a new function that adds 2 to its input, and `(*3)` multiplies its input by 3.

How to run

Environment:

- Install **GHC (Glasgow Haskell Compiler) or open an Haskell online compiler**
- The code provided in the textbook only describes the function definitions and illustrates the programming concept. To obtain actual results, an additional main function is required to execute the program and display output.

```
main = do
    print (fact 5)    -- Result:120
    print (plus2 10)  -- Result:12
    print (times3 4)  -- Result:12
```

---

3. Logic Programming (Chapter 4)

3.1 Overview

Logic programming is based on:

- Facts
- Rules (Horn clauses)
- Queries

The system automatically derives answers using inference.

3.2 Example 1: Resolution

Code

```
b :- a.
c :- b.
```

Purpose

Demonstrate logical inference using resolution.

Description

- From:

  ```
  b ← a
  c ← b
  ```

- Infer:

  ```
  c ← a
  ```

- The system replaces goals with subgoals until solved.

3.3 Example 2: GCD using Logic Programming

Code

```
gcd(u, 0, u).
gcd(u, v, w) :- not zero(v), gcd(v, u mod v, w).
```

Purpose

Compute GCD using logical inference instead of direct calculation.

Description

This program defines the GCD relation using two logical clauses. The first clause represents the base case, stating that the GCD of a number and zero is the number itself. The second clause expresses the recursive rule based on Euclid's algorithm.

Unlike procedural programming, this approach does not explicitly describe how to compute the result step by step. Instead, the programmer defines **what is true**, and the Prolog engine determines **how to derive the answer** using techniques such as **unification** and **resolution**.

When a query like `gcd(15,10,X)` is executed, the system matches it against the defined rules, recursively reducing the problem until it reaches the base case. This illustrates the declarative nature of logic programming and the separation between logic and control.

- Query:

```
gcd(15, 10, x)
```

- Result:

```
x = 5
```

---

4. Object-Oriented Programming (Chapter 5)

4.1 Overview

Object-Oriented Programming is based on:

- Classes and Objects
- Encapsulation
- Reusability

4.2 Example 1: Java Complex Class

Code

```
package numbers;
public class Complex extends Object{

public Complex(){
    this(0,0);
 }

public Complex(double realPart,double imaginaryPart){
    re=realPart;
    im=imaginaryPart;
 }

public double realPart(){
    return re;
 }

public double imaginaryPart(){
    return im;
 }

public Complex add(Complex c){
    return new Complex(this.realPart()+c.realPart(),
        this.imaginaryPart()+c.imaginaryPart());
 }

public Complex multiply(Complex c){
    return new Complex(this.realPart()*c.realPart()-
    this.imaginaryPart()*c.imaginaryPart(),
    this.realPart()*c.imaginaryPart()+
    this.imaginaryPart()*c.realPart());
 }
```

```
public String toString(){
    return this.realPart()+"+"+this.imaginaryPart()+"i";
 }

private double re,im;
}
```

Purpose

Represent complex numbers and perform operations.

Description

This class encapsulates the properties of a complex number using two private fields: the real part (`re`) and the imaginary part (`im`). By keeping these fields private, the class enforces **data encapsulation**, ensuring that the internal representation cannot be modified directly from outside the class.

Public methods such as `add()` and `multiply()` define operations on complex numbers, demonstrating how behavior is associated with data in object-oriented programming. Each operation returns a new `Complex` object, preserving immutability of existing objects.

The `toString()` method provides a human-readable representation of the object, improving usability. Overall, this example highlights key OOP principles including **encapsulation**, **abstraction**, and **reusability**.

How to run

Environment:

- Java JDK installed or use java online compiler and remember to put these code to show the result
- The code provided in the textbook only describes the function definitions and illustrates the programming concept. To obtain actual results, an additional main function is required to execute the program and display output.

```
Complex a=new Complex(1,2);
Complex b=new Complex(3,4);
System.out.println(a.add(b));
```

Expected Output:

```
4.0+6.0i
```

4.3 Example 2: C++ Complex Class

Code

```
class Complex{
    public:
    Complex(double r =0,double i =0): re(r), im(i) { }

    double realpart() const { return re; }
    double imaginarypart() const { return im; }
    Complex operator-(){ // unary minus
        return Complex( -re, -im);
    }

    Complex operator+(Complex y){
        return Complex(re+y.realpart(),im+y.imaginarypart());
    }
    Complex operator-(Complex y){ // subtraction
        return Complex(re - y.realpart(),im - y.imaginarypart
());
    }
    Complex operator*(Complex y){
    return Complex(re * y.realpart() - im * y.imaginarypart
(),im * y.realpart() + re * y.imaginarypart());
    }
    private:
```

```
    double re, im;
};
```

Purpose

Perform complex number operations using operator overloading.

Description

This class defines a `Complex` data type with two private attributes representing the real part (`re`) and imaginary part (`im`). The constructor initializes these values, allowing objects to be created easily with default or custom inputs.

The class overloads arithmetic operators such as `+`, `-`, and `*`, enabling complex number operations to be written in a natural and intuitive way (e.g., `z + w` instead of calling a function). Each operation returns a new `Complex` object, preserving the original values.

Additionally, the unary minus operator is implemented to negate a complex number. By keeping data members private and providing public methods, the class follows the principle of **encapsulation**. Overall, this example demonstrates how operator overloading and class design improve code clarity, reusability, and abstraction in object-oriented programming.

How to run

Environment:

- C++ compiler (g++, clang) save file and run through vscode or devc or open an online compiler and paste these code below to show the result
- The code provided in the textbook only describes the function definitions and illustrates the programming concept. To obtain actual results, an additional main function is required to execute the program and display output.

```
Complex z(1,2);
Complex w(-1,1);
Complex x = z + w;
```

```
cout<<"z = "<<z.realpart()<<" + "<<z.imaginarypart()<<"i\n";
cout<<"w = "<<w.realpart()<<" + "<<w.imaginarypart()<<"i\n";
cout<<"z + w = "<<x.realpart()<<" + "<<x.imaginarypart()<<"i
\n";
```

Expected Output:

```
z = 1 + 2i
w = -1 + 1i
z + w = 0 + 3i
```

🧾 5. Conclusion

This report demonstrates three different programming paradigms:

- Functional programming focuses on recursion and immutability
- Logic programming focuses on inference and rules
- Object-oriented programming focuses on objects and reuse

Each paradigm offers a different approach to problem solving and program design.